

고객상담관리 앱

AI 정리·분석 기능 상세 리포트

실제 코드(functions/api/analyze.js, functions/_lib/prompts.js, public/js/ocr.js·analysis.js·ap.js·pools.js)를 열어 확인한 내용만 담았습니다. 추측한 내용은 없습니다.

작성일	2026-08-29
대상 앱	jam-customer-console (Cloudflare Pages + D1 + R2)
AI 제공사	Anthropic Claude (Sonnet 5 / Haiku 4.5), 음성인식만 Cloudflare Workers AI(Whisper)
범위	① 고객상세 AI 정리·분석 ② 상담사례 매칭 보장분석 ③ 가입설계 분석 ④ 고객대면상담(AP) 멘트 + 참조폴 관리 AI 정리

목차

0. 전체 흐름 한눈에 보기
1. 1번 고객상세 - AI로 정리·분석 (보장금부·내보장자산 사진 정리)
2. 2번 매니저 상담 2단계 - 상담사례 매칭 보장분석
3. 3번 매니저 상담 3단계 - 가입설계 분석
4. 4번 고객대면상담(AP) - AI 대면 멘트 만들기
5. 참고: 참조폴 관리 AI로 정리 (1·2·3번에 쓰이는 참고자료를 미리 압축하는 단계)
6. 결과 화면 표시 방식 - 마크다운을 쓰는가, 글자체·크기·색상·강조는 어떻게 되는가
7. 모델·글자수 제한·이미지 규격 총정리표
8. 코드 확인 중 발견한 특이사항 (메모 미반영 등)
9. Claude Skill 제안 - 이 협업에서 반복된 패턴을 스킬로 만든다면

0 전체 흐름 한눈에 보기

네 개의 AI 버튼은 아래 순서로 이어집니다. 뒤 단계는 앞 단계의 결과를 입력값으로 그대로 가져다 씁니다.

- 1 1번 고객상세 시로 정리·분석 -> 보장 텍스트(coverageText) 생성, 고객에 저장
- 2 2번 매니저상담 상담사례 매칭 보장분석 -> 1번의 보장 텍스트 + 참조폴 자료를 분석 -> 보장분석 결과 저장
- 3 3번 매니저상담 가입설계 분석 -> 1번의 보장 텍스트 + 2번의 분석 결과 + 사전심사 + 가입설계서 사진을 분석 -> 가입설계 결과 저장
- 4 4번 고객대면상담(AP) AI 대면 멘트 만들기 -> 2·3번의 결과(요약본)를 참고자료로 다시 넣어, 상담 중 실제로 말할 멘트를 4단계로 각각 생성

2번은 1번의 결과가 없으면(보장 텍스트가 비어 있으면) 실행되지 않고, 3번은 2번을 먼저 실행해야 버튼이 활성화됩니다. 순서를 건너뛸 수 없는 구조입니다.

1 고객상세 - "AI로 정리·분석 (사진 글자 인식)"

버튼 위치: 고객상세 화면, "보장 자료 이미지" 칸 아래. 코드상 함수명 tidyCoverage() (public/js/ocr.js), 서버 mode="tidy" (functions/api/analyze.js).

무엇을 하는가

고객상세에 등록된 [보장급부] / [내보장자산] / [기타] 사진 최대 12장을 AI가 직접 눈으로 읽어(비전 분석), 종류별로 나눠 정리하고 마지막에 종합소견을 붙입니다. 결과는 화면의 "보장 자료" 텍스트칸(c-coverage)에 그대로 채워지고 고객 데이터(coverageText)로 저장됩니다. 이 값이 2·3번 분석의 재료가 됩니다.

예전에는 기기 자체 OCR 엔진(Tesseract.js)으로 먼저 글자를 추출하는 버튼(runOCR)이 따로 있었으나, 현재 화면(index.html)에는 이 버튼이 연결되어 있지 않습니다. 즉 지금 실제로 쓰이는 경로는 "AI로 정리·분석"(사진을 AI가 직접 읽는 방식) 하나뿐입니다.

입력으로 들어가는 것

- 사진: 등록된 이미지 중 앞쪽 최대 12장 (12장 넘으면 뒤는 무시)
- 고객 이름·나이(있으면 첫 줄에 반영하라고 지시)
- 직전에 "확인 필요" 질문에 사용자가 답한 내용(있으면 확정값으로 사용)

이미지 처리 방식

저장 시 리사이즈	긴 변 기준 최대 1568px, JPEG 품질 0.9 (기기에 저장되는 원본 사진)
AI 전송 시 리사이즈	긴 변 기준 최대 1568px, JPEG 품질 0.92로 다시 압축해 base64로 전송
OCR 엔진	별도 OCR 없음 - Claude 비전 모델이 이미지를 직접 읽음(표 행, 담보명, 숫자 자릿수까지)

1568px는 Anthropic이 권장하는 비전 분석 최적 해상도값입니다(과거 세션에서 이중 압축으로 글자가 뭉개지는 문제를 고치며 이 값으로 통일했습니다).

사용 모델

환경변수 VISION_MODEL이 있으면 그 모델, 없으면 기본 모델(환경변수 ANTHROPIC_MODEL, 기본값 claude-sonnet-5)을 씁니다. 즉 별도 설정이 없다면 Sonnet 5로 처리됩니다.

프롬프트 핵심 규칙 (functions/_lib/prompts.js - tidySystem)

- 사진을 먼저 한 장씩 정독하고 표·담보명·숫자·회사명을 확인한 뒤에 결과를 쓰라고 명시(추측 금지)
- 숫자는 자릿수(천/만/억)·십표를 특히 주의, 흐릿하면 "확인 필요"로 남기고 임의로 채우지 말 것
- 출력은 [보장급부] -> [내보장자산] -> [종합분석] 세 섹션·소제목 순서를 반드시 지키고 서로 섞지 말 것
- 담보는 "영역: 담보명 가입금액 (납입/만기/갱신여부)" 한 줄 압축 형식
- [종합분석]은 개조식 5~8줄: 영역별 충분/취약, 중복 영역, 부족(빈) 영역, 3000만원 ↑ 핵심담보 갱신형 위험, 우선 보완순위
- 전체 출력 목표 3500자 이내로 압축(넘으면 덜 중요한 세부부터 생략)
- 본문 끝에 "===확인필요===" 구분선을 넣고 그 아래 불확실한 항목을 질문 3개 이내로 나열(없으면 "- 없음")

max_tokens(서버가 모델에 허용하는 최대 출력 길이) = 8000 토큰

출력 형식 - 마크다운인가?

- 프롬프트가 지시하는 형식은 "개조식(하이픈 -)"과 "대괄호 [소제목]"이며, 마크다운 문법(굵게 표시, 삼, 표 등)을 쓰라는 지시도 쓰지 말라는 지시도 없습니다.

- 다만 결과가 화면의 일반 입력칸(textarea)에 그대로 채워지므로, 설정 마크다운 기호가 섞여 나와도 서식으로 "렌더링"되지 않고 글자 그대로 보입니다 - 즉 화면이 마크다운 기호로 지저분하게 깨져 보일 위험은 구조적으로 없습니다.

- 결론: 실제로도 프롬프트가 하이픈·대괄호 개조식을 강하게 지정해 마크다운 기호가 거의 나오지 않고, 설사 나오더라도 서식 없는 텍스트칸이라 깨져 보이지는 않습니다. 다만 2·3번(analyze/plan)처럼 "마크다운을 쓰지 말라"는 명시적 금지 문구는 이 프롬프트엔 없습니다.

2 매니저 상담 2단계 - "상담사례 매칭 보장분석"

코드상 함수명 runAnalysis() (public/js/analysis.js), 서버는 mode 지정 없이 기본 분기(functions/api/analyze.js 하단부).

무엇을 가져다 쓰는가 - 질문하신 항목별 확인 결과

확인 항목	결과	근거
고객 상세의 상품 관심	O 포함	customer.products로 전송
고객 상세의 상담 상황	O 포함	customer.situations로 전송
1번에서 만든 보장 텍스트	O 포함	coverageText - 핵심 입력
고객상세의 메모(c-memo)	X 미포함	어느 분석 요청에도 메모 필드를 담아 보내지 않음(코드 전체 검색 확인)
참조폴 4종 전부	3종만	상담사례(case)·에피소드(episode)·상품카달로그(catalog) 3종만 전송. 아이스브레이크(icebreak)는 전송 안 함(4번 AP에서만 사용)
매니저상담 2단계의 분석옵션	O 포함	집중분석(focusAreas)·제외(excludeAreas) 칩 선택값
참조폴 화면에서 체크(선택)한 자료	O 포함	체크된 게 있으면 그 제목을, 없으면 상품·상황 태그가 겹치는 자료를 자동으로 골라 이걸 쓰라고 지시

정리하면 - 사용자가 예로 든 조합 중 실제로 합쳐지는 것은 "1번의 정리 텍스트 + 상품관심/상담상황 + 참조폴 3종(사례·에피소드·카달로그) + 매니저상담 2단계의 분석옵션"이며, "메모"는 저장만 되지만 분석에는 반영되지 않습니다.

참조폴 전송 방식 - "고정" vs "고객 맞춤"

- 참조폴관리(관리자 화면)에 올라간 자료는 전부 "전역 고정"(globalPinned) - 고객이 바뀌어도 항상 같은 내용이 통째로 전송되고, 캐시로 처리되어 같은 내용을 반복 전송해도 비용이 절감됩니다(프롬프트 캐싱).
- 그중 "이 고객 분석에 실제로 쓸 것"은 담당자가 참조폴 화면에서 체크(선택)했으면 그 제목만, 안 했으면 상품·상황 태그가 겹치는 자료를 점수순으로 자동 선정해 제목만 별도로 엮습니다(본문 중복 전송 안 함, 비용 절감).

상담사례(case) 고정 텍스트 최대 6개
개수 제한

에피소드(episode) 고정 텍스트 최대 4개
개수 제한

상품카달로그(catalog) 전체(고정), 최대 6000자로 자름
og)

고객 맞춤 지시(사례) 체크 또는 자동선정 최대 3개(제목만)

고객 맞춤 지시(에피소드) 체크 또는 자동선정 최대 4개(제목만)

고객 맞춤 지시(카달로그) 체크 또는 자동선정 최대 6개(제목만)

사용 모델

환경변수 ANTHROPIC_MODEL(기본값 claude-sonnet-5) - 이미지가 없는 텍스트 분석이지만, 핵심 분석이라 상위 모델(Sonnet)을 그대로 사용합니다. max_tokens = 8000.

프롬프트 핵심 규칙 (analyzeSystem)

- "반드시 아래 JSON 형식 하나만 출력. 마크다운, 코드블록, 설명 문장 없이 순수 JSON만" - 명시적으로 마크다운 금지
- 분석 영역: 사망보장 / 암·3대진단 / 입원·수술 / 실손의료 / 간병·치매 / 상해·재해 중 텍스트에서 확인되는 것
- 담보가 전혀 없으면 그 영역은 "취약" + 사유에 "해당 담보 없음" 명시
- 판정마다 certainty(확정/추정/자료부족)를 반드시 구분 - "보장이 실제로 없어서 취약"과 "자료가 부족해서 알 수 없음"을 혼동 금지
- 보험료·해약환급금 등 텍스트에 없는 숫자는 지어내지 않음
- 성별이 미상이면 남성으로 임의 가정하지 않고 중립적으로 서술(앱10 세션에서 추가된 규칙)
- 가입금액 3000만원 이상 핵심 담보는 갱신형/비갱신형을 반드시 판별해 별도 소제목으로 정리(불명확하면 "확인 필요")
- 집중분석(focusAreas) 영역은 우선순위를 높이고, 제외(excludeAreas) 영역은 아예 다루지 않음

출력 JSON 구조

```
{
  "summary": "종합 요약 2~3문장",
  "areas": [{"name": "영역명", "level": "충분|보통|취약"},
  "reason": "한 줄 근거", "certainty": "확정|추정|자료부족"}],
  "priorities": ["보강 제안 1", "보강 제안 2", "보강 제안 3"],
  "detail": ["[영역별 상세]", "...", "[핵심 담보(3000만원 ↑) 갱신형 여부]",
  "...", "[종합 소견]", "..."],
  "questions": ["확실하지 않아 사용자에게 확인할 질문"]
}
```

3 매니저 상담 3단계 - "가입설계 분석"

코드상 함수명 runPlanAnalysis() (public/js/analysis.js), 서버 mode="plan".

2번 "보장분석"을 먼저 실행하고, 가입설계서 이미지를 등록해야만 버튼이 활성화됩니다.

질문하신 조합 확인 - "2번 결과 + 사전심사 + 가입설계서 이미지"가 합쳐지나요?

예, 맞습니다. 다만 실제로는 그 세 가지 외에 아래 항목도 함께 들어갑니다.

확인 항목	결과	근거
1번의 보장 텍스트(coverageText)	O	"현재 가입 내용"으로 원문 그대로 재전송
2번의 보장분석 결과	O	summary + detail 텍스트만 요약해 전송(전체 JSON이 아님)
사전심사(질병·부담보)	O	가입설계 화면 textarea 내용, 최대 4000자
가입설계서 이미지	O	최대 12장, AI가 직접 읽음(비전)
상품관심·상담상황	X	name/age/region/gender/job만 담아 보냄 - 상품관심·상담상황은 제외
참조폴 3종(사례·에피소드·카달로그)	O	2번과 동일한 구조로 전송(개수 제한은 표 참고)
매니저상담 2단계의 분석옵션(집중/제외)	O	고객에 저장된 focusAreas/excludeAreas 그대로 재사용
메모(c-memo)	X	2번과 동일하게 미포함

참조폴 중 "상담사례(case)"는 시스템 프롬프트(planSystem) 안에서 활용법이 따로 설명되어 있지 않습니다 - 코드상 데이터는 함께 전송되지만(analyze.js), 프롬프트 지침 문장에는 에피소드·카달로그만 이렇게 활용하라고 적혀 있고 상담사례에 대한 지침은 없습니다. 실제 분석 품질에 큰 영향은 없어 보이지만 프롬프트-데이터 불일치로 볼 수 있는 부분입니다.

이미지 처리 방식

1번과 동일: 긴 변 최대 1568px, 전송 품질 0.92 JPEG. OCR 없이 AI가 사진을 직접 읽습니다.

사용 모델

환경변수 VISION_MODEL(없으면 기본 모델) - 사진을 읽어야 하므로 1번과 같은 비전 경로. max_tokens = 8000.

프롬프트 핵심 규칙 (planSystem)

- 가입설계서 사진을 분석 전에 먼저 한 장씩 정독, 숫자 자릿수·유효표 주의(1번과 동일한 정독 규칙)
- 사전심사가 있으면 반드시 반영 - 부담보 걸린 담보는 충분히 보장됨으로 판단하지 말 것, 질병 이력은 향후 가입 제한 가능성과 연결해 해석
- "반드시 아래 JSON 하나만 출력하세요. 마크다운·설명 없이 순수 JSON만" - 명시적 마크다운 금지
- shortfallRate(잔여 부족율, 0~100 정수) - 가입설계 반영 후에도 남아있는 부족 보장 비율
- planDetail은 [현재 보장 부족점] -> [이 설계가 채운 부분] -> [남아있는 부족·추가 보완] -> [추가 제안(연금 등)] 순서 고정, 네 부분 중복 금지
- 확실하지 않은 값(가입금액·담보명 등)은 questions에 담고 추측 금지
- 성별 미상 시 중립 서술(2번과 동일 규칙)

출력 JSON 구조


```
{  
  "shortfallRate": 40,  
  "summary": "이 설계가 무엇을 채우고 무엇이 남는지 2~3문장",  
  "planDetail": ["[현재 보장 부족점]", "...", "[이 설계가 채운 부분]", "...",  
    "[남아있는 부족·추가 보완]", "...", "[추가 제안(연금 등)]", "..."],  
  "questions": ["확인이 필요한 짧은 질문"]  
}
```

4 고객대면상담(AP) - "AI 대면 멘트 만들기"

코드상 함수명 genApScripts() (public/js/ap.js), 서버 mode="ap". "매니저 상담" 3단계를 마친 뒤 "고객대면상담" 버튼으로 진입하는 별도 화면입니다.

구조 - 4단계, 단계별로 AI를 각각 따로 호출

단계	실제로 넘기는 참고자료(material)
아이스브레이크(ice)	참조폴의 icebreak 자료 중 선택/자동선정 최대 6개(제목+본문)
보장분석 설명(cov)	2번 보장분석 결과 중 가장 최근 1건 - summary+영역판정+우선순위+상세를 이어붙인 텍스트
에피소드(epi)	참조폴의 episode 자료 중 선택/자동선정 최대 6개(제목+본문)
가입설계 제안(plan)	3번 가입설계 결과 중 가장 최근 1건 - summary+잔여부족율+planDetail을 이어붙인 텍스트

2·3번의 결과를 다시 보낼 때는 이미지나 원본 JSON 전체가 아니라, 이미 화면에 쓰던 텍스트(summary·영역·우선순위 등)를 한 번 더 이어붙인 요약 문자열입니다. material은 서버에서 최대 12,000자로 자릅니다.

사용자가 체크한 단계만 순서대로 하나씩 개별 API 호출로 생성합니다(한 번에 4개를 합쳐 보내지 않음) - 화면에는 "생성 중... (2/4 - 보장분석 설명)" 식으로 진행률이 표시됩니다.

사용 모델

환경변수 TIDY_MODEL(기본값 claude-haiku-4-5-20251001) - 4개 버튼 중 유일하게 가장 저렴한 Haiku 모델을 씁니다. max_tokens = 1500.

프롬프트 핵심 규칙 (apSystem)

- "설계사가 읽고 그대로 말할 수 있는 실제 대화체 멘트"를 만들라고 지시 - 분석 리포트가 아니라 대사
- 팔려고 밀어붙이지 말고 공감·사실 중심, 고객이 스스로 필요를 느끼게 유도
- 참고 자료에 없는 사실은 지어내지 않음
- 성별 미상이면 중립적으로 작성(2·3번과 동일 규칙)
- 분량 600자 이내, 인사말·사족·메타설명 없이 멘트 본문만

화면 표시

생성된 멘트는 각 단계 화면에 "AI 대면 멘트 (설계사 참조용)" 라벨과 함께 일반 텍스트로 그대로 표시됩니다(esc() 함수로 HTML 특수문자만 이스케이프 - 마크다운 렌더링 없음, 있는 그대로의 문장).

5 참고 - 참조폴 관리 "AI로 정리" (1·2·3번의 재료를 미리 압축)

관리자 화면(관리자 화면 -> 참조폴 관리)에서 상담사례·에피소드·아이스브레이크·상품카달로그 원문을 넣고 누르는 버튼입니다. 여기서 미리 압축해 둔 결과가 위 2·3번 분석에 "참조폴 자료"로 반복 재사용됩니다.

입력: 텍스트(.txt) 또는 음원(짧은 것). 음원은 Cloudflare Workers AI의 whisper-large-v3-turbo로 먼저 받아쓰기한 뒤, 그 결과를 원문 뒤에 이어붙여 같은 파이프라인으로 정리합니다(20MB 넘으면 서버가 차단).

사용 모델

환경변수 TIDY_MODEL(기본값 claude-haiku-4-5-20251001). max_tokens = 3200. 원문은 최대 20,000자까지만 사용.

출력 JSON 구조 및 글자수 제한

```
{
  "titleKeyword": "핵심 키워드 (최대 40자)",
  "summary": "목차식 구성 요약 (최대 1200자)",
  "keyContent": "실제 상담에 쓸 핵심 내용, 개조식 (최대 4000자)",
  "tags": ["매칭용 키워드 3~8개"]
}
```

이 중 keyContent가 이후 2·3번 분석 때 그대로 재사용되는 값입니다 - 여기서 미리 압축해둘수록 매 분석 때 드는 토큰(비용)이 줄어듭니다.

음성변환 원문은 오타자·오인식이 섞일 수 있어, 보험 용어·숫자가 문맥상 어색하면 임의로 고치지 말고 "(확인 필요)"를 붙이라는 규칙이 있습니다(앱9 세션에서 추가).

6 결과 화면 표시 방식 - 마크다운·글자체·크기·색상·강조

마크다운을 쓰는가

- 2번 보장분석, 3번 가입설계, 참조폴 정리: 프롬프트가 "마크다운 없이 순수 JSON만"을 명시적으로 강제 - AI가 구조화된 데이터(JSON)만 반환하고, 화면은 그 데이터를 앱이 직접 카드·배지·글머리표로 조립해 그림니다. 마크다운이 끼어들 자리가 없는 구조입니다.
- 1번 정리·분석(tidy), 4번 AP 멘트: 자유 텍스트를 반환하지만 결과가 textarea 또는 이스케이프된 일반 텍스트로만 표시되어, HTML/마크다운 서식이 애초에 렌더링되지 않습니다.
- 결론: 어떤 버튼도 마크다운 기호가 화면에 서식 깨짐으로 보이는 구조가 아닙니다. 다만 1·4번은 프롬프트에 "마크다운 금지"를 명시하지 않았다는 점에서 2·3번보다는 형식 통제가 느슨합니다.

2·3번 결과 화면의 실제 표시 규칙 (public/js/core.js, analysis.js, ap.js)

기본 글자체	Apple SD Gothic Neo / Noto Sans KR / Malgun Gothic 등 기기 시스템 한글 글꼴(웹폰트 없음)
본문 글자 크기	13.5~15px, 줄간격(line-height) 1.65~1.75
기본 글자색	진한 남색 계열 #1c2530(제목) / #4a5560(본문) / #8892a0(보조설명)
개조식 변환	summary 등 줄글 문장을 문장 단위로 쪼개 각 줄 앞에 가운뎃점을 붙여 불릿처럼 표시(bulletize 함수)
숫자·긍정 키워드 강조	금액·기간·비율 및 충분/확보/적정/우수/비갱신형 등 -> 파란색 굵게(#1a56db)
위험·부정 키워드 강조	부족/위험/미흡/취약/시급/공백/미가입/해지/갱신형/확인 필요 등 -> 빨간색 굵게(#c0392b)
영역별 판정 색	충분=초록(#2f7d4f) / 보통=황토색(#9a7714) / 취약=빨강(#a33636), 왼쪽 색 띠(3px)로도 표시
신뢰도(certainty) 배지	확정/추정/자료부족 3색 별도 배지로 표시(자료가 부족해 모르는 것과 실제 취약을 구분)
가입설계 부족율	큰 숫자(26px, 굵게)+막대그래프, 60% ↑ 빨강 / 30~59% 황토 / 30%미만 초록
소제목([영역별 상세] 등)	대괄호째로 굵게 표시, 그 아래 항목들을 가운뎃점으로 나열

AP 멘트(4번) 화면 표시

"이렇게 말해보세요"라는 라벨이 붙은 카드(ap-script 클래스) 안에 굵은 서식 없이 텍스트 그대로 표시됩니다 - 강조 색상 규칙(위 표)은 2·3번 결과 화면에만 적용되고 AP 멘트에는 적용되지 않습니다.

7 모델 · 글자수 제한 · 이미지 규격 총정리

버튼	서버 mode	모델(환경변수)	max_tokens	이미지	출력형식
1번 AI로 정리·분석	tidy	VISION_MODEL (기본 Sonnet 5)	8000	최대 12장 1568px q0.92	자유 텍스트 (3500자 목표)
2번 상담사례 매칭 보장분석	(기본)	ANTHROPIC_MODEL (기본 Sonnet 5)	8000	없음(텍스트만)	JSON (마크다운 금지 명시)
3번 가입설계 분석	plan	VISION_MODEL (기본 Sonnet 5)	8000	최대 12장 1568px q0.92	JSON (마크다운 금지 명시)
4번 AP 대면 멘트	ap	TIDY_MODEL (기본 Haiku 4.5)	1500	없음	자유 텍스트 (600자 이내)
참조폴 AI 정리	organize_pool	TIDY_MODEL (기본 Haiku 4.5)	3200	없음(음원 가능)	JSON (keyContent 4000자)
참조폴 요약(보조)	summarize	TIDY_MODEL (기본 Haiku 4.5)	2600	없음	자유 텍스트 (약 2000자)

비용 절감 장치: 프롬프트 캐싱

- 참조폴(관리자 화면) 자료·시스템 프롬프트처럼 매번 똑같이 반복 전송되는 긴 글에 "캐시해도 된다" 표시(cache_control)를 붙여, 짧은 시간 안에 같은 내용이 다시 오면 Anthropic이 다시 읽지 않고 캐시에서 가져와 입력 비용을 1/10로 낮춥니다.
- 고객마다 달라지는 부분(선택된 참조자료 제목, 고객 정보 등)은 캐시 대상이 아니라 매번 새로 계산되지만, 제목만 담아 분량이 작아 비용에 미치는 영향이 거의 없습니다.

8 코드 확인 중 발견한 특이사항

- 고객상세의 "메모" 칸은 어떤 AI 분석 요청에도 전송되지 않습니다. 저장만 되지만 1·2·3·4번 어디에도 반영되지 않는 필드입니다.
- 참조물은 4종(상담사례·에피소드·아이스브레이크·상품카달로그)이 있지만, 2번 보장분석·3번 가입설계에는 아이스브레이크를 뺀 3종만 들어갑니다. 아이스브레이크는 4번 고객대면상담의 1단계에서만 쓰입니다.
- 3번 가입설계 요청에는 상담사례(case) 자료도 함께 전송되지만, 3번의 시스템 프롬프트(planSystem)에는 상담사례를 이렇게 활용하라는 지침 문장이 없습니다(에피소드·카달로그는 지침이 있음). 데이터는 가는데 활용 지시가 빠져 있는 상태입니다.
- 예전에 있던 기기 자체 OCR 버튼(Tesseract.js 기반 runOCR 함수)은 현재 화면(index.html)에 연결된 버튼이 없어 실제로는 쓰이지 않는 죽은 코드로 보입니다. 지금은 "AI로 정리·분석"(AI 비전) 경로 하나만 실제로 동작합니다.
- 1·4번은 자유 텍스트 출력이라 "마크다운을 쓰지 말라"는 명시적 금지 문구가 없습니다(2·3번에는 있음). 다만 결과가 서식 없는 화면 요소에 표시되는 구조라 실제 화면에서 문제가 되지는 않습니다.

이 리포트는 2026-08-29 기준 코드 상태를 그대로 반영합니다. 이후 코드가 바뀌면 이 문서 내용도 그만큼 달라집니다.

9 Claude Skill 제안 - 이 협업에서 반복된 패턴을 스킬로 만든다면

이 문서까지 이어진 여러 세션(앱8~앱12) 동안 jam님과 반복해서 확인된 소통 방식·요구사항을 정리했습니다. jam-customer-console 폴더 안에는 이미 이런 정리를 시도한 기록이 두 개 더 있었습니다(SKILL_jam작업방식.md, AI기능_상세리포트_클로드스킬.docx). 계정에 저장된 스킬 2개(jam-dev-collaboration-style, jam-ai-feature-collaboration)까지 포함해 네 자료를 모두 대조해서, 겹치는 내용은 하나로 합치고 지금도 유효한 내용만 아래에 정리했습니다. 오래된 배포 방식(Netlify 등) 설명처럼 지금과 다른 내용은 제외했습니다.

이미 저장·기록되어 있는 자료 4개

jam-dev-collaboration-style (스킬)	쉬운 한국어·직설적 소통, "모르면 모른다" 원칙, 답하기 전 실제 파일·배포 사이트 직접 확인, 결정 필요 지점은 먼저 물어보기, 파일 작업 순서(수정→검증→전달), 세션 간 인계.
jam-ai-feature-collaboration (스킬)	AI 출력 포맷을 구체적 숫자로 지정하는 성향, AI 기능은 일반 버튼과 시각적으로 구분, 비용을 항상 먼저 생각, 관리자 중심·공용자원 절감 구조 선호, 불필요한 UI 제거, 에러는 원인까지 설명.
SKILL_jam 작업방식.md (폴더 내 문서)	위 두 스킬보다 더 세밀한 누적 기록. 개발 진행 방식(단계별 진행·매 편집 후 검증), UI/UX 취향(3D 버튼·진행률%·히스토리 형식), 보험 도메인 규칙, JSON 파싱 등 반복된 기술 함정까지 포함.
AI기능_상세 리포트_클로드스킬.docx (폴더 내 문서)	위와 비슷한 취지의 별도 정리본. 비용을 "남은 잔액이 줄어드는" 방식으로 보여주는 것을 선호한다는 점, 실제 보험 업무 자료 흐름(스캔 이미지로 받는 자료가 많음)까지 포함.

소통·의사결정 습관

- 모호한 요청("정리해줘")도 되묻기 전에 합리적인 기본안을 먼저 제시하고, 필요하다면 그 다음에 조정하는 방식을 선호합니다.
- 선택지를 주면 그중 "추천"을 고르는 편입니다 — 장단점 비교와 명확한 추천을 함께 제시하는 것이 잘 맞습니다.
- 결정이 필요한 지점은 짧은 질문(버튼형 선택)으로 먼저 확인하되, 이미 정한 것은 다시 묻지 않는 것을 선호합니다.
- 화면 캡처로 상태를 공유하고, 그 화면을 근거로 원인·다음 단계를 짚어주길 기대합니다.
- 스펙을 정확한 숫자로 정하는 것을 좋아합니다(예: "사진 12장으로 제한", "5000자 이내로 압축").
- 여러 작업을 한 메시지에 몰아서 요청하는 경우가 많습니다. 항목별로 쪼개 순서대로 처리하고, 각각 커밋해서 진행상황을 남기는 방식이 잘 맞습니다.
- 같은 방법이 반복 실패하면 "이번엔 될 것"이라고 넘기지 않고, 정직하게 상황을 인정한 뒤 다른 방법이나 대안을 제시해야 합니다.

코드·개발 작업 습관

- 큰 변경은 단계별로 쪼개서 진행하는 것을 선호합니다. 한 번에 너무 많이 바꾸면 앱이 깨진 적이 있어, 단계마다 확인하고 다음으로 넘어가는 흐름을 신뢰합니다.
- 코드를 고치면 문법 검증(node --check)과 실제 동작 확인까지 끝나야 "완료"로 봅니다. 중간에 멈추지 않습니다.
- 답변 전에는 실제 파일·배포 사이트 상태를 직접 확인해야 합니다(캐시된 기억으로 판단 금지) — 여러 작업 세션을 병행해서 쓰기 때문에, 다른 세션에서 이미 바뀐 내용이 이 세션엔 반영 안 되어 있을 수 있습니다.
- 작업이 길어지거나 세션이 바뀔 때는 진행상황·결정사항을 문서로 남겨(claude_현재상태.md 등), 다음 세션이 바로 이어받을 수 있게 합니다.
- 여러 파일이 같이 바뀔 때는 바뀐 파일을 전부 명확히 알려주고, 무엇을 커밋해야 하는지 짚어줍니다.

UI/UX 취향 (반복적으로 요구된 것)

- AI가 대신 처리해주는 기능은 일반 버튼과 시각적으로 구분되는 전용 스타일을 씁니다.
- 삭제처럼 되돌릴 수 없는 동작에는 확인창을 기본으로 둡니다(이번 세션의 "모든 삭제 버튼에 확인창 추가"가 대표적).
- 이미 통과한 인증은 다시 요구하지 않습니다 — 불필요한 반복 확인 단계를 싫어합니다.
- 안 쓰는 UI 요소나 중복되는 화면은 과감히 제거하고, 겹치는 흐름은 하나로 통합합니다(이번 세션의 홈 화면·매니저상담 단계 통합).
- 모바일 사용을 기본으로 두고 설계합니다 — 한 화면에 한 단계씩, 다음·이전 버튼을 명확히.
- 비용은 "남은 잔액이 줄어드는" 방식으로 보여주는 것을 선호합니다. 계속 누적으로 올라가는 방식은 헛갈려 합니다.

AI 분석 도메인 규칙 (보험 상담 특성상 항상 지킬 것)

- 회사명·숫자·건수는 절대 추측하지 않고, 불명확하면 "확인 필요"로 표기합니다.
- 보장급부와 내보장자산은 서로 섞지 않고 엄격히 구분해서 정리합니다.
- 가입금액이 큰 핵심 담보는 갱신형·비갱신형 여부를 반드시 판별해서 표기합니다.
- 성별이 확인되지 않으면 특정 성별로 임의로 가정하지 않고 중립적으로 서술합니다.
- 참조폴(상담사례·에피소드·상품카달로그)을 보장분석·가입설계 양쪽 분석에 함께 참고합니다.

이번 세션에서 새로 드러나 아직 정리되지 않은 것

- jam-customer-console 전용 커밋 절차: 이 폴더를 GitHub Desktop과 동시에 쓰기 때문에, 코드 수정 후 .git/index.lock·.git/HEAD.lock이 있으면 삭제 대신 이름변경(이 환경은 파일 삭제가 막혀 있음) → 커밋 → 커밋 직후 생긴 새 lock도 다시 이름변경, 순서를 매번 지켜야 합니다. 프로젝트 고유의 절차라 위 네 자료 어디에도 없습니다.
- 삭제 버튼은 예외 없이 확인창을 넣는 것이 기본값이라는 점이 이번에 명확한 규칙으로 확인됐습니다. 다음부터 새 삭제 기능을 만들 때는 처음부터 confirm()을 포함해서 만들면 됩니다.
- "물어보기"와 "지시하기"를 구분하는 규칙: 논의 중인 아이디어(질문형 문장, "~게 아닐까요", "~이 맞나요")는 코드 작업 없이 설명만 드리고, 실제 실행 지시(명령형 문장, 또는 "해줘"·"합쳐줘" 같은 분명한 표현)일 때만 구현에 들어가는 것으로 정리하면, 논의 단계에서 성급하게 코드를 고치는 실수를 줄일 수 있습니다.

이 섹션은 제안입니다. 네 자료(스킬 2개 + 문서 2개)에 흩어진 내용을 하나로 정리한 것이며, 실제 스킬 파일(.skill / SKILL.md)로 업데이트할 부분은 사용자가 요청하면 별도로 반영합니다.